

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Algorytmy i Złożoność Obliczeniowa

Projekt 1 — Analiza algorytmów sortowania

Ocena docelowa: 5.0

Autor: Ivan Taborskykh 288347

Prowadzący: mgr inż. Damian Mroziński

Data: 11.05.2026

Spis treści

1	Wstęp	2
2	Opis zaimplementowanych algorytmów	2
2.1	Bucket Sort	2
2.2	Quick Sort	2
2.3	Shell Sort	3
3	Opis struktur danych	3
4	Metodologia badań	3
5	Wyniki badań	4
5.1	Badanie α — wpływ parametrów algorytmu	4
5.1.1	QuickSort: strategię wyboru pivotu	4
5.1.2	ShellSort: formuły sekwencji odstępów	5
5.2	Badanie A — wpływ rozmiaru zbioru	6
5.3	Badanie B — wpływ rozkładu danych wejściowych	7
5.4	Badanie C — wpływ typu danych	8
5.5	Badanie Ω — wpływ struktury danych	9
6	Wnioski	10

1 Wstęp

Celem projektu była implementacja trzech algorytmów sortowania — *Bucket Sort*, *Quick Sort* oraz *Shell Sort* — i zbadanie ich wydajności w zależności od kilku czynników: parametrów wewnętrznych algorytmu, rozmiaru danych, rozkładu danych wejściowych, typu przechowywanych elementów oraz zastosowanej struktury danych.

Wszystkie algorytmy zaimplementowano w C++ wersji 17 z użyciem szablonów (`template`), co umożliwiło jednolite testowanie na tablicach dynamicznych, listach jednokierunkowych, listach dwukierunkowych, stosach oraz drzewach BST. Czas sortowania mierzono wyłącznie dla operacji sortowania (bez wczytywania danych i bez alokacji pamięci; wyjątkiem jest drzewo BST, gdzie alokacja węzłów jest nieodłączną częścią operacji wstawiania) przy użyciu `std::chrono::high_resolution_clock::now()`, a wynik rzutowano na `std::chrono::microseconds`. Każdy wariant powtórzono 67 razy, a jako wynik przyjęto średnią arytmetyczną.

Link do GitHub [link](#)

2 Opis zaimplementowanych algorytmów

2.1 Bucket Sort

Bucket Sort (sortowanie przez podział na wiadra) działa w trzech fazach. Najpierw wyznacza zakres wartości w tablicy wejściowej ($[\min, \max]$) i tworzy $k = \lfloor \sqrt{n} \rfloor$ pustych wiadr, gdzie n to liczba elementów. Następnie każdy element jest przypisywany do wiadra proporcjonalnie do jego wartości w tym zakresie. Na koniec każde wiadro jest sortowane osobno algorytmem wstawiania (*insertion sort*), a wyniki scalane w jedną posortowaną tablicę.

Złożoność całkowita wynosi $O(n + n^2/k)$: dystrybucja elementów to $O(n)$, a przy jednorodnym rozkładzie danych każde wiadro zawiera n/k elementów, więc łączny koszt insertion sort wynosi $k \cdot O((n/k)^2) = O(n^2/k)$. Przy zastosowanym wyborze $k = \lfloor \sqrt{n} \rfloor$ upraszcza się to do $O(n \cdot \sqrt{n}) = O(n^{3/2})$ [1].

Algorytm obsługuje wyłącznie typy liczbowe (`int`, `float`, `double`, `unsigned long`). W przypadku łańcuchów znaków sortowanie zamiast Bucket Sort wykonuje osobną implementację bucket sort opartą na wartości ASCII pierwszego znaku.

2.2 Quick Sort

Quick Sort (sortowanie szybkie) w zaimplementowanej wersji używa dwupunktowego schematu partycji: pivot jest przestawiany na koniec podciągu, po czym dwa wskaźniki zbiegają się ku sobie, zamieniając elementy poza miejscem. Aby uniknąć przepełnienia stosu wywołań przy głębokiej rekurencji (np. dla danych posortowanych), zastosowano implementację iteracyjną z jawnym stosem.

Dostępne są cztery strategie wyboru elementu osiowego (*pivot*):

- **Losowy** (-p 0) — pivot wylosowany z aktualnego podciągu;
- **Lewy** (-p 1) — pierwszy element podciągu;

- **Prawy** (-p 2) — ostatni element podciągu;
- **Środkowy** (-p 3) — element ze środka podciągu.

Złożoność oczekiwana wynosi $O(n \log n)$, natomiast pesymistyczna $O(n^2)$ — ta ostatnia pojawia się, gdy pivot jest zawsze elementem skrajnym, co zdarza się na danych posortowanych lub odwrotnie posortowanych przy wyborze pivota z lewego lub prawego krańca [1].

2.3 Shell Sort

Shell Sort (sortowanie Shella) to uogólnienie sortowania przez wstawianie, gdzie elementy porównuje się i przestawia w dużych odstępach, stopniowo zmniejszając odstęp do 1. Dzięki temu elementy dalekie od swoich docelowych pozycji są szybko przesuwane w odpowiednim kierunku.

Zaimplementowano dwie sekwencje odstępów:

- **Formuła Shella** (-e 0) — $h = \lfloor n/2^k \rfloor$; złożoność $O(n^2)$ w pesymistycznym przypadku;
- **Formuła Knutha** (-e 1) — $h_1 = 1$, $h_{k+1} = 3h_k + 1$; złożoność $O(n^{3/2})$.

3 Opis struktur danych

Wszystkie zaimplementowane struktury danych są dynamiczne i oparte na szablonach, bez użycia standardowych kontenerów STL. Każda struktura udostępnia metody `toArray()` i `fromArray()`, co pozwala na jednolite sortowanie: dane są konwertowane do tablicy, sortowane, a następnie przepisywane z powrotem.

Tablica dynamiczna Ciągły blok pamięci; dostęp losowy $O(1)$.

Lista jednokierunkowa Węzły z polem `next`; dostęp sekwencyjny, bez dostępu losowego.

Lista dwukierunkowa Węzły z polami `next` i `prev`; umożliwia iterację w obu kierunkach.

Stos Struktura LIFO oparta na liście jednokierunkowej; operacje `push/pop`.

Drzewo BST Drzewo przeszukiwania binarnego; wstawianie elementów buduje drzewo, a przejście *in-order* zwraca je posortowane. Czas sortowania obejmuje budowę drzewa i jego przejście.

4 Metodologia badań

Środowisko testowe: MacBook Air (Apple M3), system macOS 14.6 (Sonoma), kompilator Apple Clang 16.0.0 (arm64). Program kompilowano z flagami `-Wall -Wextra -Werror`.

Do obsługi parametrów wejściowych użyto biblioteki dostarczonej przez prowadzącego (`libparameters.so` dla systemu Linux). Badania przeprowadzono na systemie macOS, gdzie użyto odpowiednika tej biblioteki w formacie `libparameters.dylib`, udostępnionego przez znajomego. Obie wersje biblioteki są funkcjonalnie równoważne i zgodne interfejsem. Plik `CMakeLists.txt` automatycznie wybiera właściwą wersję w zależności

od platformy docelowej: kod poprawnie kompiluje się i działa zarówno na systemie Linux (z `libparameters.so`), jak i na macOS (z `libparameters.dylib`).

Dla każdego wariantu testu wykonywano 67 niezależnych pomiarów. Dane generowano od nowa przed każdym pomiarem. Mierzono wyłącznie czas operacji sortowania. Po każdym pomiarze weryfikowano poprawność sortowania; błędne pomiary były pomijane, jednak w żadnym przypadku nie wystąpiły błędy sortowania. Wyniki agregowano jako: średnia, minimum i maksimum czasu wykonania.

Rozmiary zbiorów w badaniu A: 10 000, 25 000, 50 000 i 100 000 elementów. Badania B, C, Alpha i Omega przeprowadzono dla $n = 25\,000$ elementów (wartość środkowa z badania A).

5 Wyniki badań

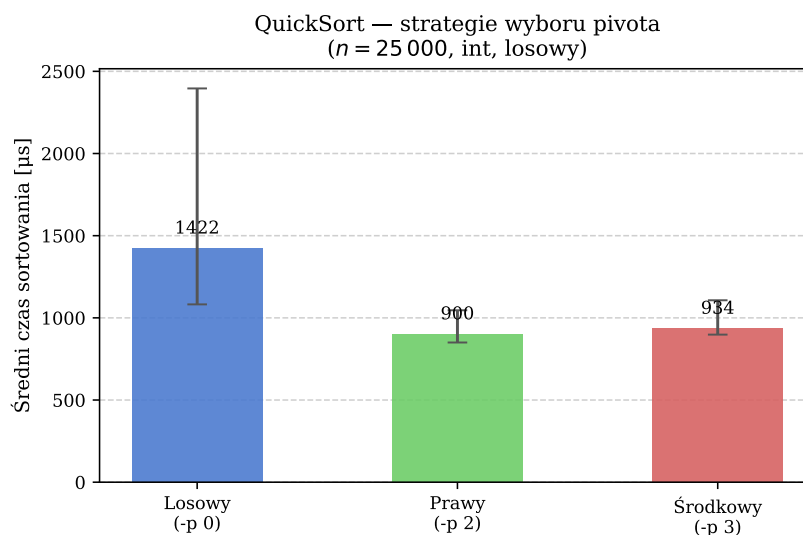
5.1 Badanie α — wpływ parametrów algorytmu

5.1.1 QuickSort: strategie wyboru pivota

W tym badaniu porównano trzy strategie wyboru elementu osiowego dla zbioru 25 000 losowych liczb całkowitych na tablicy. Wyniki zestawiono w tabeli 1.

Tabela 1: Czas sortowania QuickSort w zależności od strategii pivota ($n = 25\,000$, dane losowe, typ int).

Strategia pivota	Średnia [μs]	Min [μs]	Max [μs]
Losowy (-p 0)	1 422	1 082	2 396
Prawy kraniec (-p 2)	900	850	1 046
Środkowy (-p 3)	934	898	1 107



Rysunek 1: Średni czas sortowania QuickSort dla trzech strategii wyboru pivota (słupki błędu: min/max).

Pivot losowy okazał się ok. 58% wolniejszy od pivotu prawego i ok. 52% wolniejszy od środkowego. Wynika to z narzutu wywołania `std::uniform_int_distribution` przy każdym kroku podziału — przy $n = 25\,000$ takich wywołań jest rzędu n , co daje mierzalny narzut łączny.

Istotna jest też wyższa zmienność: odchylenie standardowe dla pivotu losowego wyniosło $\approx 321\ \mu\text{s}$, podczas gdy dla pivotu prawego i środkowego wynosiło zaledwie $\approx 32\ \mu\text{s}$. Wysoka wariancja wynika z tego, że losowy pivot może czasem trafić blisko krańca podciągu i wygenerować niezbalansowany podział — choć przy danych losowych zdarza się to rzadko.

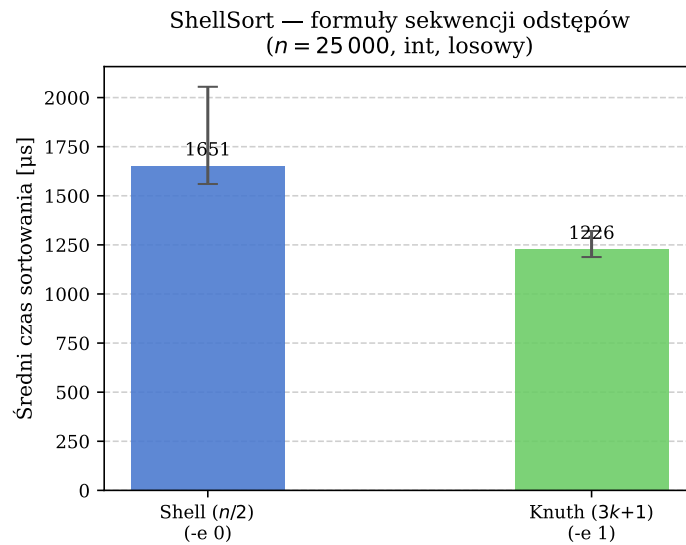
Na danych losowych pivot prawy i środkowy osiągają zbliżone czasy i są bardziej przewidywalne, więc losowość nie przynosi tu żadnej korzyści praktycznej.

5.1.2 ShellSort: formuły sekwencji odstępów

Tabela 2 i rysunek 2 przedstawiają porównanie dwóch sekwencji odstępów Shell Sorta.

Tabela 2: Czas sortowania ShellSort w zależności od formuły odstępów ($n = 25\,000$, dane losowe, typ int).

Formuła	Średnia [μs]	Min [μs]	Max [μs]
Shell ($n/2$)	1 651	1 560	2 055
Knuth ($3k+1$)	1 226	1 188	1 321



Rysunek 2: Średni czas sortowania ShellSort dla dwóch sekwencji odstępów (słupki błęd: min/max).

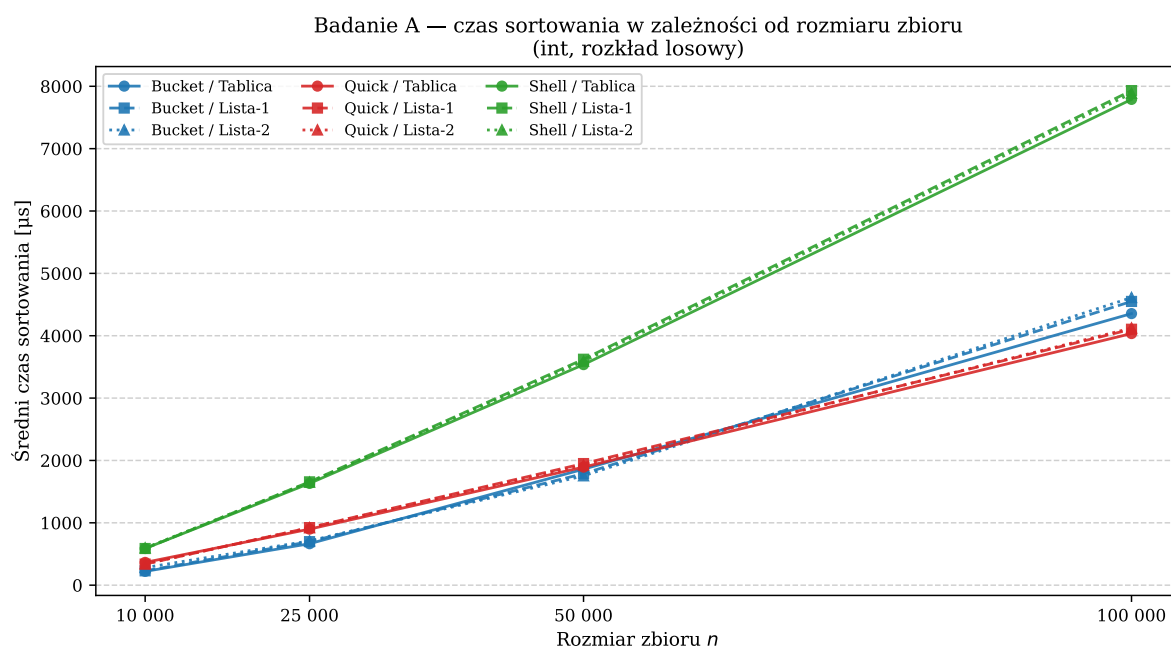
Formuła Knutha okazuje się o ok. 26% szybsza od oryginalnej formuły Shella. Wynika to z lepszych własności sekwencji $1, 4, 13, 40, \dots$: liczba faz jest zbliżona, ale każda faza lepiej redukuje liczbę inwersji, ponieważ wartości odstępów są dobrane tak, aby nie były wielokrotnościami poprzednich — co minimalizuje powtarzanie podobnych porównań.

5.2 Badanie A — wpływ rozmiaru zbioru

Badanie A mierzyło czas sortowania dla czterech rozmiarów zbiorów i wszystkich dziewięciu kombinacji algorytm $\times$ struktura. Dane były losowe, typ int. Wyniki dla tablic i list pokazują rysunek 3, a szczegółowe wartości tabela 3.

Tabela 3: Średni czas sortowania [μs] w zależności od algorytmu, struktury i rozmiaru zbioru.

Algorytm	Struktura	$n = 10\,000$	$n = 25\,000$	$n = 50\,000$	$n = 100\,000$
Bucket Sort	Tablica	221	665	1 860	4 355
	Lista jednokierunkowa	230	698	1 788	4 550
	Lista dwukierunkowa	282	710	1 753	4 612
Quick Sort	Tablica	365	899	1 893	4 035
	Lista jednokierunkowa	338	923	1 950	4 104
	Lista dwukierunkowa	341	921	1 924	4 123
Shell Sort	Tablica	588	1 632	3 537	7 790
	Lista jednokierunkowa	588	1 657	3 623	7 931
	Lista dwukierunkowa	596	1 650	3 590	7 886



Rysunek 3: Średni czas sortowania w zależności od rozmiaru zbioru dla trzech algorytmów i trzech struktur liniowych.

Wyniki ujawniają kilka ciekawych zależności. Przy $n = 10\,000$ Bucket Sort jest najszybszy ($\approx 221 \mu s$), następnie Quick Sort ($\approx 365 \mu s$) i Shell Sort ($\approx 588 \mu s$). Bucket Sort zachowuje przewagę aż do $n \approx 50\,000$, gdzie czasy Bucket i Quick Sort wyrównują się (odpowiednio $\approx 1\,860$ i $\approx 1\,893 \mu s$). Przy $n = 100\,000$ Quick Sort ($\approx 4\,035 \mu s$) wyprzedza Bucket Sort ($\approx 4\,355 \mu s$), a Shell Sort pozostaje wyraźnie najwolniejszy ($\approx 7\,790 \mu s$).

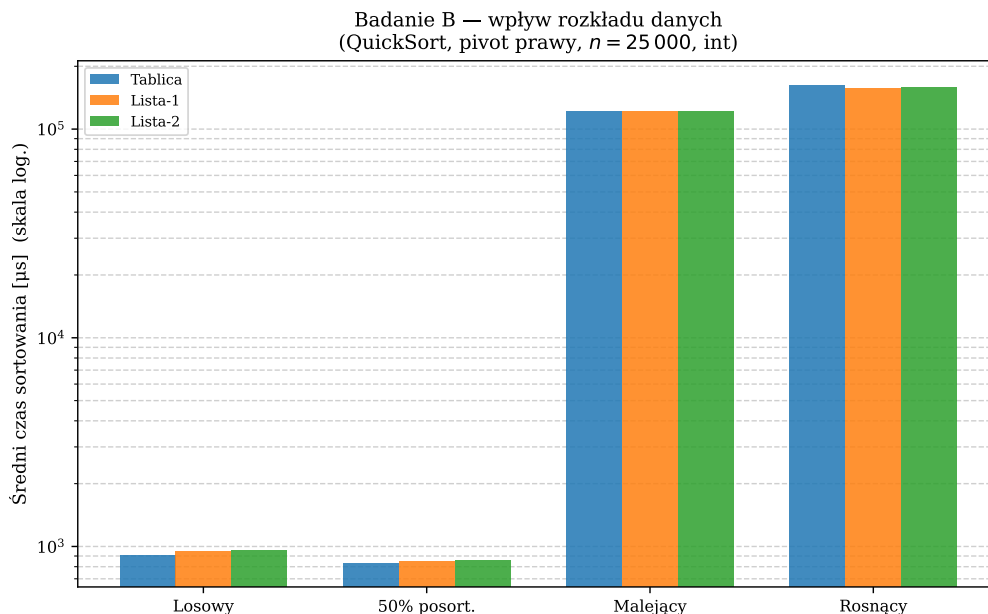
Różnica między typem struktury (tablica, lista jedno- i dwukierunkowa) jest minimalna dla wszystkich algorytmów. Wynika to ze świadomego wyboru projektowego: algorytmy Quick Sort i Shell Sort wymagają dostępu losowego do elementów (indeksowania), którego listy wiązane nie zapewniają. Z tego powodu przed sortowaniem dane z listy są kopiowane do tablicy pomocniczej, algorytm operuje na tablicy, a wynik jest przepisywany z powrotem do listy. Sama operacja kopiowania ma złożoność $O(n)$ i nie jest wliczana do mierzonego czasu. Badanie A potwierdza ten efekt: czas sortowania praktycznie nie zależy od struktury, co jest bezpośrednim następstwem tego podejścia.

5.3 Badanie B — wpływ rozkładu danych wejściowych

W badaniu tym użyto Quick Sort z pivotem prawym (-p 2) dla $n = 25\,000$ elementów int na trzech strukturach liniowych. Zbadano cztery rozkłady: losowy, 50% posortowany, rosnący i malejący. Wyniki (dla tablicy) zawiera tabela 4, a wizualizację rysunek 4.

Tabela 4: Czas sortowania QuickSort (pivot prawy, tablica, $n = 25\,000$, int) dla różnych rozkładów danych.

Rozkład	Średnia [μs]	Min [μs]	Max [μs]
Losowy	910	875	962
50% posortowany (<code>ascending50</code>)	833	771	983
Malejący (odwrotny)	121 561	118 086	142 534
Rosnący (posortowany)	163 235	154 910	227 224



Rysunek 4: Średni czas sortowania QuickSort dla różnych rozkładów danych (oś Y w skali logarytmicznej).

Wyniki potwierdzają znany fakt teoretyczny: Quick Sort z pivotem z prawego krańca osiąga pesymistyczną złożoność $O(n^2)$ na danych posortowanych i odwrotnie posortowanych. Przy

$n = 25\,000$ czas rośnie do $\approx 163\,000\ \mu\text{s}$ dla rozkładu rosnącego — blisko 180-krotnie więcej niż dla danych losowych.

Warto zauważyć, że dane malejące ($\approx 121\,500\ \mu\text{s}$) są ok. 26% szybsze od danych rosnących ($\approx 163\,000\ \mu\text{s}$), mimo że oba przypadki są teoretycznie $O(n^2)$. W obu przypadkach pivot prawy jest zawsze elementem skrajnym (maksimum lub minimum), lecz wzorzec dostępu do pamięci jest inny: przy danych rosnących lewy wskaźnik przemierza całą partycję bez zamiany, podczas gdy przy danych malejących prawy wskaźnik szybko natrafia na właściwy element. Ta różnica w liczbie porównań i zamian między wskaźnikami wpływa na mierzalną różnicę czasu wykonania.

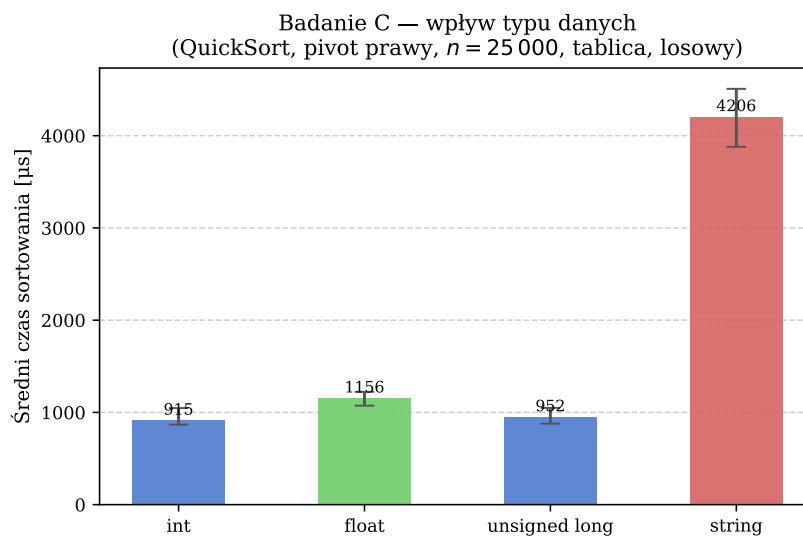
Ciekawa jest wartość dla danych 50% posortowanych: wynosi zaledwie $\approx 833\ \mu\text{s}$, czyli nawet *mniej* niż dla danych czysto losowych ($\approx 910\ \mu\text{s}$). Wynika to stąd, że elementy częściowo ułożone w porządku wymagają mniejszej liczby przestawień podczas partycjonowania, co redukuje stały czynnik złożoności mimo identycznej asymptotyki $O(n \log n)$.

5.4 Badanie C — wpływ typu danych

Badanie C sprawdziło wpływ rozmiaru i kosztu porównania elementów na czas sortowania Quick Sort (pivot prawy, $n = 25\,000$, tablica, dane losowe). Wyniki w tabeli 5 i na rysunku 5.

Tabela 5: Czas sortowania QuickSort w zależności od typu danych ($n = 25\,000$, tablica, dane losowe).

Typ danych	Średnia [μs]	Min [μs]	Max [μs]
int	915	867	1 047
unsigned long	952	878	1 049
float	1 156	1 073	1 223
string	4 206	3 879	4 508



Rysunek 5: Średni czas sortowania QuickSort dla różnych typów danych.

Typy `int` i `unsigned long` osiągają prawie identyczny czas (odpowiednio 915 i 952 μs), co nie dziwi — oba mają taki sam rozmiar i podobny koszt porównania na poziomie instrukcji maszynowych. `float` jest wolniejszy o ok. 26%, co wynika z dodatkowego kosztu generowania losowych wartości zmiennoprzecinkowych (normalizacja do zakresu) oraz nieco wyższej latencji porównań `float` względem `int`.

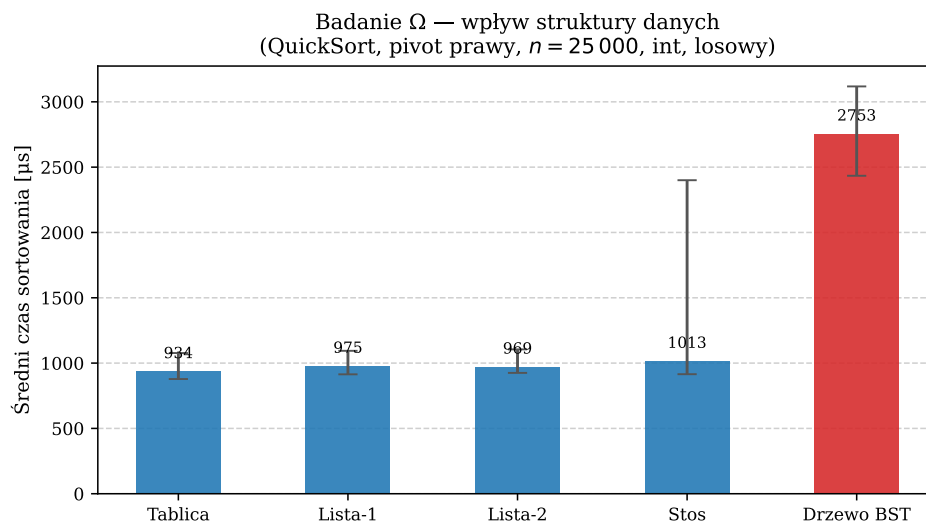
Najbardziej rzucający się w oczy jest wynik dla łańcuchów znaków: `string` jest ponad 4,5-krotnie wolniejszy od `int`. Każde porównanie dwóch ciągów wymaga iteracji po znakach do momentu znalezienia różnicy, a ponieważ losowe łańcuchy mają pewną minimalną długość (1–10 znaków), koszt jednego porównania jest znacznie wyższy niż porównanie liczby. Łącznie Quick Sort wykonuje $O(n \log n)$ porównań, więc ten narzut kumuluje się wyraźnie.

5.5 Badanie Ω — wpływ struktury danych

Badanie porównało pięć struktur danych: trzy liniowe (tablica, lista jedno- i dwukierunkowa), stos oraz drzewo BST, wszystkie z Quick Sort (pivot prawy, $n = 25\,000$, typ `int`, dane losowe). Wyniki zestawiono w tabeli 6 i na rysunku 6.

Tabela 6: Czas sortowania Quick Sort dla różnych struktur danych ($n = 25\,000$, `int`, dane losowe).

Struktura danych	Średnia [μs]	Min [μs]	Max [μs]
Tablica	934	878	1 077
Lista jednokierunkowa	975	914	1 094
Lista dwukierunkowa	969	925	1 105
Stos	1 013	915	2 400
Drzewo BST	2 753	2 434	3 118



Rysunek 6: Średni czas sortowania Quick Sort dla pięciu struktur danych.

Cztery struktury liniowe (tablica, dwie listy, stos) uzyskały niemal identyczne czasy w przedziale 934–1 013 μs . Potwierdza to obserwację z badania A: ponieważ Quick Sort

wymaga dostępu losowego, wszystkie struktury liniowe są przed sortowaniem kopiowane do tablicy pomocniczej. Mierzony czas obejmuje wyłącznie operację sortowania na tej tablicy — narzut samego kopiowania ($O(n)$, poza pomiarem) jest marginalny. Wynik badania potwierdza zatem, że przy takim podejściu typ struktury liniowej nie ma wpływu na czas sortowania.

Drzewo BST wyróżnia się wynikiem $\approx 2\,753\ \mu\text{s}$ — prawie trzykrotnie dłużej niż tablica ($\approx 934\ \mu\text{s}$). W tym przypadku czas mierzony obejmuje budowę drzewa przez n wstawiń (każde $O(\log n)$ dla zrównoważonego drzewa, lecz bez równoważenia BST może degenerować się do $O(n)$) oraz przejście *in-order* (liniowe $O(n)$). Narzut alokacji węzłów w pamięci dynamicznej i mniejsza lokalność danych (każdy węzeł to osobna alokacja na sterpie) tłumaczą ten znaczący wzrost.

6 Wnioski

Przeprowadzone badania pozwoliły sformułować kilka wniosków dotyczących praktycznych własności zaimplementowanych algorytmów.

Wybór algorytmu. Dla danych losowych i typów numerycznych Bucket Sort był najszybszy dla $n \leq 25\,000$, a Quick Sort (pivot prawy lub środkowy) wyprzedził go dopiero przy $n = 100\,000$. Shell Sort konsekwentnie plasował się na trzecim miejscu — jego złożoność jest wyższa niż $O(n \log n)$ Quick Sorta (formuła Shella ma pesymistyczne $O(n^2)$, formuła Knutha — $O(n^{3/2})$).

Wybór pivota. Strategia losowa jest ok. 58% wolniejsza od deterministycznych (prawy lub środkowy pivot), co wynika z narzutu wywołania generatora liczb pseudolosowych przy każdym podziale. Dodatkowo charakteryzuje się wyższą wariancją czasu (odch. std. $\approx 321\ \mu\text{s}$ wobec $\approx 32\ \mu\text{s}$ dla pozostałych strategii). Na danych losowych pivot prawy lub środkowy jest szybszy i bardziej przewidywalny.

Rozkład danych. Quick Sort z pivotem prawym jest ekstremalnie wrażliwy na dane posortowane — czas rośnie z $\approx 0,9\ \text{ms}$ do $\approx 163\ \text{ms}$, czyli wzrost prawie 180-krotny. Dane malejące są ok. 26% szybsze od rosnących, mimo identycznej złożoności $O(n^2)$, co wynika z różnic w wzorcu dostępu do pamięci podczas partycjonowania. W zastosowaniach produkcyjnych wybór pivota ze środka lub podejście *median-of-three* jest niezbędny do zachowania efektywności.

Typ danych. Typy całkowite (`int`, `unsigned long`) są porównywalne. `float` jest wolniejszy o ok. 26% z powodu kosztu operacji zmiennoprzecinkowych. Łańcuchy znaków powodują ponad 4-krotny wzrost czasu ze względu na droższe porównania leksykograficzne.

Struktura danych. Quick Sort i Shell Sort wymagają dostępu losowego, dlatego struktury liniowe (lista, stos) są przed sortowaniem kopiowane do tablicy — koszt kopiowania $O(n)$ nie jest wliczany do czasu badania. Skutkiem tego wyboru projektowego jest brak mierzalnej różnicy między strukturami liniowymi. Drzewo BST jest prawie trzykrotnie wolniejsze z powodu kosztu n wstawiń i rozłożonej alokacji pamięci powodującej chybienia cache.

Sekwencja Knutha. Shell Sort z sekwencją Knutha ($3k + 1$) okazał się systematycznie szybszy ($\approx 26\%$) od oryginalnej formuły Shella ($n/2$). Dla większych zbiorów różnica ta może być jeszcze bardziej widoczna.

Literatura

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022. Rozdziały 7 (QuickSort) i 8.4 (Bucket Sort).